

SCALE

Scalable Crowd Behavior via Cached Language-model Experiences — Real-Time
LOD-Aware Rendering of Semantically Diverse Crowds

CGVR (CS373IA) Research Proposal

2026

SCALE — Scalable Crowd Behavior via Cached Language-model Experiences

Real-time LOD-aware rendering of semantically diverse, LLM-behavior-driven crowds

Publication-oriented project concept, system design, technical approach, evaluation protocol, and paper skeleton

Note on scope: This proposal is designed to carry genuine, substantial weight in two courses concurrently — Computer Graphics and Virtual Reality (CS373IA) and Parallel Architecture and GPU Programming (CS372IA). Sections 1–6 and 8–14 establish the graphics/crowd-simulation contribution; **Section 7 is the parallel-computing implementation that makes that contribution’s real-time scale claim actually true**, not a decorative addition — see Section 7.2 for an explicit mapping to the CS372IA syllabus.

1. Research Problem

Two active research threads in computer graphics currently do not talk to each other.

Thread A — LLM-driven crowd behavior. Recent work (2025–26) shows that giving crowd agents LLM-generated reasoning produces far more believable, socially plausible behavior than classical rule-based models (social force, pure ORCA) — agents that queue, defer, browse, and react to context the way real people do, not just avoid collisions. But every one of these systems reports the same bottleneck: calling an LLM live for each agent, each timestep, is computationally and financially prohibitive once you scale past a few dozen agents.

Thread B — scalable crowd rendering. A separate, older lineage of graphics research (impostors, GPU instancing, level-of-detail animation, vertex-animation-texture skinning) solves how to *render* thousands of independently animated agents in real time — but this work assumes each agent’s behavior comes from a cheap, low-variety source (a small set of looping animations), because that assumption is what makes the rendering optimizations tractable.

The unaddressed question: what happens to a scalable crowd-rendering pipeline when the underlying behavior is no longer low-variety — when each agent’s action is drawn from a rich, semantically diverse, LLM-generated repertoire? Does rendering-side scalability survive contact with behavior-side richness, and if not, what does it cost to get both at once?

Research question: Can a cached library of LLM-generated behavior primitives, combined with an LOD-aware rendering pipeline, deliver crowds that are both semantically diverse *and* real-time at scale (hundreds

to low thousands of agents) — without the live per-agent LLM cost that makes Thread A’s approach unscalable?

Hypotheses: - **H1:** Live per-agent LLM querying becomes non-real-time (drops below interactive frame rates) well before a cached-primitive approach does, at the same agent count. - **H2:** A cached-primitive approach preserves measurably higher behavioral diversity than a classical finite-state-machine crowd AI baseline, at comparable frame rate. - **H3:** LOD-aware rendering (distance-based animation-tier and mesh/impostor switching) recovers most of the frame-rate cost of behavioral diversity, compared to naive full-detail rendering of every agent regardless of distance. - **H4:** The behavior primitives, when cross-checked against real pedestrian trajectory statistics (ETH/UCY), produce group/avoidance/pacing patterns consistent with real crowd data — i.e., diversity is not achieved at the cost of physical plausibility.

2. Why This Is Relevant to CGVR

This project sits squarely in computer graphics’ crowd-simulation-and-rendering lineage (Reynolds’ boids, Helbing’s social force model, RVO/ORCA, impostor-based crowd rendering, GPU-instanced animation) while updating it with the current LLM-agent moment. It is explicitly positioned as **infrastructure for VR training simulations** (evacuation training, public-space navigation training, museum/crowd-education scenarios) — the kind of large-scale, behaviorally rich crowd scene a VR training application would need — without requiring a headset to build or evaluate it: the system is built and benchmarked on desktop, with VR deployment named as the direct next step.

Position the paper as:

“We identify and quantify the tension between LLM-driven behavioral diversity and real-time crowd rendering, and present a cached-behavior-primitive architecture with LOD-aware rendering that recovers scalability without sacrificing diversity, evaluated for large-scale VR training scenarios.”

Avoid claiming to invent LLM-crowd behavior, ORCA collision avoidance, or crowd-rendering LOD techniques — all three exist independently and are cited as prior work. The contribution is identifying the gap between them and building/measuring the combined system.

3. Proposed Contribution

- **Cached behavior-primitive architecture:** An offline-generated, embedding-indexed library of LLM-authored behavior descriptions (intent + locomotion + gesture), retrieved at runtime via lightweight similarity search instead of a live LLM call per agent per frame.
- **Behavior-to-LOD coupling:** A rendering pipeline where an agent’s animation/mesh fidelity tier (full skeletal mesh, GPU-instanced simplified skeleton, VAT-baked impostor) is chosen jointly by camera distance *and* current behavior complexity, rather than distance alone.
- **Scalability benchmark:** A systematic sweep of frame rate, draw calls, and CPU/GPU cost across agent counts (10 □ 2000+) and across four conditions: classical FSM crowd AI, live-LLM-per-agent, cached-primitive without LOD, and cached-primitive with LOD — isolating exactly where the AI cost and the rendering cost each bite.
- **Diversity-plausibility validation:** A behavioral-diversity metric (entropy over selected primitives across agents/time) cross-checked against ETH/UCY real pedestrian trajectory statistics, so richness is validated against reality rather than asserted.
- **GPU-parallel retrieval and collision-avoidance pipeline:** A batched, GPU-parallel embedding-retrieval kernel for behavior-primitive selection (the system’s other per-agent bottleneck alongside collision avoidance), implemented and benchmarked across serial CPU, multithreaded CPU, CUDA, and OpenACC, and integrated with an adapted GPU-parallel ORCA solver — making the full

pipeline's real-time claim (Section 8) something the parallel-computing implementation actually earns, not something asserted.

4. Proposed System Architecture

```
OFFLINE (build time)
LLM behavior-primitive authoring
(intent + locomotion + gesture text descriptions,
e.g. "hurries past, glancing at phone",
"pauses to let a group pass, then resumes")
|
v
Embedding index (primitive library)
|
=====
RUNTIME (per agent, per decision tick)
|
v
Agent context (goal, nearby agents, zone semantics)
|
v
Embedding similarity lookup ---> selected behavior primitive
|
v
Local collision avoidance (RV02 / ORCA)
-- converts primitive intent into a collision-free velocity --
|
v
Distance-to-camera + behavior complexity
|
v
LOD selector
|
+-----+-----+
| | |
v v v
Full skeletal mesh GPU-instanced VAT-baked impostor
(near agents, (mid-distance, (far agents,
rich gesture set) simplified skeleton) baked loop only)
| | |
+-----+-----+
|
v
Unity Renderer (desktop, URP)
|
v
Benchmark logger: FPS, draw calls,
CPU/GPU time, agent count, diversity entropy
|
+----- feedback: agent re-queries context each tick ----->
```

5. Technology Stack

Layer	Recommended technology
Engine	Unity 6, Universal Render Pipeline (URP)
Local collision avoidance	RVO2-Unity (Nebukam ORCA, job-system multithreaded) or RVO2-CS port
Crowd rendering optimization	GPU-instanced skinned animation / Vertex Animation Texture (VAT) baking; Unity Asset Store LOD+impostor bundle as a starting integration, customized for behavior-tier coupling
Behavior-primitive generation	Any capable LLM API, used offline only (not at runtime) to author and paraphrase the primitive library
Embedding + retrieval	Small local sentence-embedding model + cosine-similarity nearest neighbor (no network call at runtime)
GPU-parallel benchmarking (CS372IA)	CUDA via Numba-CUDA or CuPy (Python prototyping); OpenACC for the directive-based comparison; Nsight Compute (or equivalent) for occupancy/warp-efficiency profiling
Real-time GPU deployment	HLSL compute shaders (Unity’s native compute pathway) — the production port of the validated CUDA/OpenACC decomposition
Benchmark data	ETH/UCY public pedestrian trajectory datasets (no new data collection)
Logging	CSV/JSON — FPS, draw calls, agent count, condition, diversity entropy per run
Analysis	Python, Pandas, NumPy, SciPy, Matplotlib

6. Technical Approach

6.1 Offline behavior-primitive library

Author (with LLM assistance, human-curated) a library of 40–80 short behavior primitives spanning locomotion intent (hurry, browse, wait, follow, avoid), social context (defer to group, join queue, react to obstacle), and paired gesture/animation tags. Each primitive is embedded once, offline. This is a one-time cost — it never runs during simulation.

6.2 Runtime retrieval

Each agent, on its decision tick (not every render frame — decision ticks can run at a lower rate than rendering, e.g. 2–5 Hz, further saving cost), builds a short context description from its current state (nearby agents, goal, zone semantics) and retrieves the nearest primitive by embedding similarity. This replaces a live LLM call with a vector lookup — the core mechanism that removes Thread A’s scaling bottleneck.

6.3 Local collision avoidance

The selected primitive’s intent (e.g., “defer to group”) is converted into a preferred velocity/goal offset, then handed to an ORCA solver for collision-free local navigation. This guarantees physical plausibility regardless of which primitive was chosen — the classical layer is a safety net under the semantic layer.

Important scoping note: GPU-parallelized ORCA is not new — Charlton et al. (2019) already demonstrated a CUDA implementation reaching roughly 30× speedup over multi-core CPU and simulating 100,000+ agents at 60 FPS. Do not present a from-scratch GPU-ORCA solver as this project’s novel contribution — a reviewer will find that prior work immediately. Instead, **adapt the published GPU-ORCA approach** as the collision-avoidance layer (cite it directly), and place the project’s parallel-computing novelty on the behavior-retrieval kernel (Section 7), which has no comparable prior art since LLM-driven behavior-primitive libraries for crowds are themselves only months old.

6.4 Behavior-coupled LOD rendering

Standard crowd-rendering LOD switches purely on camera distance. Here, the LOD selector also considers behavior complexity: an agent currently performing a distinctive gesture-bearing primitive is more likely to be perceptually salient even at moderate distance, and is bumped up a tier; an agent idling or in a simple repeated-loop primitive is dropped to impostor tier sooner. This is the technical crux that makes the rendering pipeline genuinely respond to the AI layer rather than sit beside it decoratively.

7. GPU-Parallel Implementation: A Parallel Computing Case Study (CS372IA Integration)

This section gives the project genuine, substantial weight in Parallel Architecture and GPU Programming (CS372IA) — not a decorative CUDA mention, but a from-scratch parallel-algorithm design and benchmarking exercise built around the system’s two actual computational bottlenecks. Both are chosen because they are the same two things that make Section 8’s real-time crowd-scale claim possible in the first place: this is the enabling implementation work under the CGVR-facing evaluation, not a separate bolt-on.

7.1 Why these two kernels

- **Collision avoidance (ORCA):** each agent solves a small linear program against nearby agents’ velocity obstacles — independent per agent once a spatial-partitioning step identifies neighbors. A textbook data-decomposition problem.
- **Behavior-primitive retrieval:** each agent computes similarity between its current context embedding and every primitive in the library — a per-agent dense vector operation, batchable into one matrix-matrix product across all agents. A textbook SIMD/data-level-parallelism problem.

7.2 Mapping to CS372IA syllabus units

CS372IA Unit	Project component
Unit I — Performance measurement, quantitative principles	Speedup and efficiency reporting across all implementations relative to the serial baseline; Amdahl’s-Law-style analysis of the retrieval/collision-avoidance share of the frame budget
Unit II — Multiprocessors and thread-level parallelism	CPU multithreaded baseline (Unity Job System / C# parallel-for) as the shared-memory-multiprocessor reference point
Unit III — Data-level parallelism, SIMD, GPU architectures, CUDA	Core kernel implementation: explicit thread/block/grid mapping for the retrieval kernel; shared memory for per-block neighbor lists; constant/texture memory for the read-only primitive-embedding table

CS372IA Unit	Project component
Unit IV — Parallel algorithm design, decomposition, load balancing	Spatial-partitioning decomposition for collision-avoidance neighbor queries; load-balancing analysis given non-uniform agent density across a scene (dense queue vs. sparse plaza)
Unit V — OpenACC, applied GPU programming case study	Directive-based OpenACC port of the retrieval kernel, compared against the hand-written CUDA version on both performance and development effort — directly matching the syllabus’s own “Applied GPU Programming: Application Case Study” framing (pyTorch/TensorFlow/Numba)

7.3 Implementations compared

1. **Serial CPU** — reference baseline for speedup calculations.
2. **Multithreaded CPU** (Unity Job System, or OpenMP-style if prototyped standalone) — shared-memory thread-level parallelism.
3. **CUDA** — prototyped and benchmarked in Python via Numba-CUDA or CuPy for fast iteration and clean profiling, with explicit thread/block/grid tuning and memory-hierarchy optimization (shared memory for neighbor lists, constant memory for the embedding table).
4. **OpenACC** — directive-based port of the same retrieval kernel, for a direct performance-vs-productivity comparison against the hand-written CUDA version.

The collision-avoidance kernel adopts the published GPU-ORCA approach (Section 6.3) rather than re-implementing it independently; the retrieval kernel is implemented from scratch across all four approaches above, since it has no existing reference implementation to adapt.

7.4 Metrics

Category	Metrics
Performance	Speedup vs. serial baseline, wall-clock time vs. agent count, memory bandwidth utilization
GPU-specific	Occupancy / warp efficiency (via Nsight Compute or equivalent profiler)
Load balancing	Variance in per-thread/per-block work given non-uniform agent density (Unit IV concept, directly measurable)
Productivity	Lines of code and development time, CUDA vs. OpenACC, for the same kernel

7.5 Deployment path

The CUDA/OpenACC benchmarking work is done as a standalone Python/C++ module — this is the CS372IA-facing deliverable, independently gradable and independently reportable regardless of how the CGVR half turns out. For the real-time Unity system, the validated parallel decomposition is re-implemented as an HLSL compute shader (Unity’s native GPU-compute pathway, necessary since collision-avoidance and retrieval results must feed directly into the frame that renders them). Reporting that the compute-shader port reproduces the CUDA-benchmarked scaling behavior is itself a small but genuine result: it shows the decomposition designed and validated in the CS372IA-style benchmarking work transfers cleanly to a production real-time engine.

8. Evaluation Design

Conditions (4, compared pairwise across the same scenes): 1. Classical baseline — finite-state-machine crowd AI (idle/walk/avoid), naive rendering, no LOD. 2. Live-LLM-per-agent — one real LLM call per agent per decision tick, naive rendering. 3. Cached-primitive, no LOD — this project’s behavior layer, full-detail rendering for all agents regardless of distance. 4. Cached-primitive + LOD (full system) — this project’s complete pipeline.

Scenario set: 3 scenes of increasing scale and complexity (open plaza, transit-hub with queuing, evacuation corridor with a bottleneck) — reuse the low/medium/high complexity pattern common in this kind of benchmark, swept across agent counts 10, 50, 200, 500, 1000, 2000.

Metrics:

Category	Metrics
Rendering performance	FPS, draw calls, GPU/CPU frame time, memory
AI cost	Decision-tick latency, total behavior-layer compute time per frame
Behavioral diversity	Shannon entropy of selected primitives across agents and time window
Plausibility	Comparison of emergent group-avoidance/pacing statistics against ETH/UCY trajectory data (distributional comparison, not per-point ADE/FDE matching — this is a sanity check, not a prediction-accuracy claim)

Primary result figure: FPS vs. agent count, one line per condition — showing where condition 2 (live LLM) falls off a cliff, where condition 3 recovers AI-side scalability but not rendering-side, and where condition 4 (full system) holds real-time frame rates furthest into the agent-count sweep. This single figure carries the paper’s core claim.

Optional stretch (cut first if time is short): a small, informal preference pilot — a handful of volunteers (not a formal human-subjects study; anonymous ranking of short pre-rendered video clips, no personal data collected) indicating which condition’s crowd looks most natural. Treat this as illustrative, not a statistically powered claim, and check with your instructor whether your institution requires even lightweight ethics sign-off for this — if there’s any doubt, drop it and rely on the entropy/ETH-UCY validation instead.

9. Recommended Development Strategy

This is now a ~14-week plan (extended from the original 12) to accommodate the parallel-computing work in Section 7 as its own real phase rather than squeezing it into the margins. If your semester is shorter, cut in this order: ETH/UCY validation first, then the OpenACC comparison (keep CUDA — dropping OpenACC loses coursework breadth but not the core parallel-computing result).

Phase	Weeks	Deliverable
1. Setup	1–2	Unity project scaffolding; integrate RVO2-Unity (CPU reference); build one baseline scene (classical FSM crowd, no LOD)

Phase	Weeks	Deliverable
2. Behavior library	3	Author and embed the 40–80 primitive library offline; build runtime embedding-retrieval lookup (CPU version first)
3. Behavior-ORCA integration	4	Wire primitive intent output into ORCA preferred-velocity input; verify collision-free plausible motion
4. GPU kernel development (CS372IA core)	5–7	Serial CPU, multithreaded CPU, CUDA (Numba/CuPy), and OpenACC implementations of the retrieval kernel; adapt the published GPU-ORCA approach; benchmark all four (Section 7)
5. Live-LLM baseline	8	Build condition 2 (live-LLM-per-agent) for the CGVR-side comparison — small agent counts only, since it's expected to fail early
6. Rendering pipeline	9–10	Integrate GPU-instanced/VAT rendering and LOD/impostor system; couple LOD tier selection to behavior complexity; port the validated GPU kernels to Unity compute shaders
7. Full benchmark harness	11	Automated sweep across agent counts/scenes/conditions (Section 8); logging to CSV; entropy metric implementation
8. ETH/UCY validation (optional/stretch)	12	Load public trajectory data; compute distributional comparison — only if ahead of schedule (cut first if not)
9. Analysis & write-up	12–14	Statistical/plots (the FPS-vs-agent-count figure and the parallel-implementation speedup figure, first), paper draft(s), submission prep

10. What Would Constitute a Strong Result?

The interesting outcome is not “our system is fastest” in isolation — it’s the **shape of the trade-off curve**: exactly how much behavioral diversity costs in frame budget, and how much of that cost the LOD coupling recovers, and at what agent-count threshold each condition stops being real-time. A result showing the cached+LOD system holds real-time performance to, say, 5–10× the agent count that the live-LLM baseline can sustain — while retaining measurably higher diversity than the classical FSM baseline — is a complete, publishable, and honest story even if the numbers aren’t dramatic. A finding that LOD coupling matters more at high agent counts than low ones (or vice versa) is also a legitimate, reportable result — don’t force a single universal claim if the data says otherwise.

11. Publication Positioning

Avoid: *“We built a smarter, faster crowd simulator.”* Prefer: *“We identify and quantify the previously unmeasured trade-off between LLM-driven behavioral diversity and real-time crowd rendering, and present a cached-primitive, LOD-coupled architecture that substantially recovers scalability, validated against real pedestrian trajectory statistics and positioned for large-scale VR training scenarios.”*

Primary targets: ACM SIGGRAPH/Eurographics Symposium on Computer Animation (SCA), MIG (Motion in Games), Eurographics short papers/posters — all directly publish crowd-simulation-and-rendering systems work of this shape. **Backup:** Computer Graphics Forum, IEEE VR poster (given the explicit VR-training framing). **Guaranteed fallback:** arXiv preprint on completion regardless of review outcome.

Consider splitting the output in two. Section 7’s GPU-parallel implementation is substantial and independently reportable — rather than forcing one paper to carry both courses, a legitimate option is a graphics-facing paper/poster (crowd behavior + rendering, as above) plus a separate short parallel-computing report or poster (the CUDA/OpenACC/multithreaded-CPU comparison on the retrieval and collision-avoidance kernels) for a systems/HPC-facing outlet or simply as the CS372IA course deliverable. This gives you two smaller, more certain outputs instead of one larger combined one, and each course gets a deliverable that stands on its own.

12. Paper Skeleton

1. **Introduction** — motivate the gap between LLM-crowd-behavior research and scalable crowd rendering; state the research question and contribution.
 2. **Related Work** — LLM-driven crowd/agent behavior (SenseWalk, CrowdVLA, Emergent Crowds Dynamics); classical crowd simulation (social force, RVO/ORCA); scalable crowd rendering (impostors, GPU instancing, VAT, LOD).
 3. **System Design** — architecture (Section 4), behavior-primitive library and retrieval (6.1–6.2), ORCA integration (6.3), behavior-coupled LOD rendering (6.4).
 4. **Parallel Implementation** — GPU-parallel retrieval kernel and adapted GPU-ORCA collision avoidance, four-way implementation comparison, CS372IA-mapped metrics (Section 7).
 5. **Evaluation** — conditions, scenarios, metrics, benchmark procedure (Section 8).
 6. **Results** — FPS-vs-agent-count figure (the core result) □ parallel-implementation speedup/occupancy comparison □ diversity entropy comparison □ ETH/UCY plausibility check □ any scene-dependent or agent-count-dependent findings.
 7. **Discussion** — where the AI-side cost dominates vs. where the rendering-side cost dominates; what the CUDA-vs-OpenACC comparison implies for development effort at this scale; design implications for future LLM-driven crowd systems; limitations (2D collision avoidance layer, offline-authored primitive library rather than fully generative at runtime, desktop-only evaluation).
 8. **Limitations and Future Work** — runtime/dynamic primitive generation, VR headset deployment and in-headset perceptual study, richer 3D navigation (multi-floor, dynamic obstacles).
 9. **Conclusion**
-

13. Risks and Mitigations

Risk	Mitigation
GPU-instanced skinned animation / VAT baking takes longer than expected	Fall back to simpler capsule/primitive avatars with baked loop animations for the LOD study — the scalability <i>trend</i> is the result, not photorealistic characters
Live-LLM-per-agent baseline (condition 2) is slow/costly to run at scale	Only run it up to the agent count where it visibly falls off real-time (a few hundred); no need to push it to 2000 agents — the falling-off point <i>is</i> the finding
ETH/UCY validation proves harder to interpret than expected	Treat as optional/stretch (Section 8); the core FPS/diversity result stands without it
Embedding retrieval produces repetitive-feeling behavior despite a large primitive library	Add lightweight temporal variety constraints (avoid repeating an agent's last N primitives) — a small, cheap fix, not a redesign
CUDA-to-Unity native plugin integration underestimated (this is the single biggest schedule risk in the whole project)	Never attempt a native CUDA plugin bridge into Unity. Do all CUDA/OpenACC benchmarking as a standalone Python/C++ module outside Unity (this is the CS372IA-gradable deliverable on its own); port only the <i>validated decomposition</i> to an HLSL compute shader for the real-time system — a much smaller integration surface
Reimplementing GPU-ORCA from scratch takes too long	Adapt the published Charlton et al. approach and the existing RVO2-Unity codebase rather than deriving it independently — the parallel-computing novelty budget is spent on the retrieval kernel, not the collision-avoidance kernel
OpenACC comparison adds scope without adding research novelty	Keep it regardless — it is expected coursework rigor for CS372IA Unit V, not a novelty claim. If time is very tight, it is the first parallel-computing item to cut, after ETH/UCY

14. Literature Starting Points

- Reynolds, “Flocks, Herds, and Schools” (foundational boids model).
- Helbing & Molnár, Social Force Model (1995).
- van den Berg et al., Reciprocal Velocity Obstacles / Optimal Reciprocal Collision Avoidance (ORCA) — RVO2 Library.
- Ulicny et al., “Crowd Simulation for Interactive Virtual Environments and VR Training Systems”; “Impostors and Pseudo-Instancing for GPU Crowd Rendering.”
- NVIDIA GPU Gems 3, Chapter 2: “Animated Crowd Rendering.”
- SenseWalk: Agent-Based Semantic Trajectory Simulation Powered by LLMs in Zoned Environments (2025).
- CrowdVLA: Embodied Vision-Language-Action Agents for Context-Aware Crowd Simulation (2026).
- Emergent Crowds Dynamics from Language-Driven Multi-Agent Interactions (2025).
- Recent LoD-representation comparisons for animated characters (Mesh vs. Impostor vs. NeRF vs. 3D Gaussian) as a possible future-work extension.
- Charlton et al., “Fast Simulation of Crowd Collision Avoidance” (arXiv 1908.10107, 2019) — GPU-parallel ORCA, ~30x speedup over multi-core CPU, 100,000+ agents at 60 FPS. This is the approach Section 6.3/7.3 adapts rather than reinvents; cite directly and do not claim GPU-ORCA as a novel contribution.
- Li & Shih, “An Early Performance Comparison of CUDA and OpenACC”; Hoshino et al., “CUDA vs OpenACC: Performance Case Studies with Kernel Benchmarks and a Memory-Bound CFD Application”

— methodological reference for Section 7.3–7.4’s comparison design (expect roughly 37% shorter development time for OpenACC and up to ~50% lower raw performance versus hand-tuned CUDA, as a sanity check against your own measurements).